Universidade Estadual de Campinas

Faculdade de Engenharia Elétrica e de Computação

Diagnóstico UDS — Sistema AEB

Modelo MIL com Security Access Criptográfico

ISO 14229-1 (Unified Diagnostic Services)

Rodrigo Faguetti

Março de 2026

Arquivos: `AEB_UDS.slx` • `AEB_UDS_build.m` • `AEB_UDS_scenarios.m`

Sumário

1 Introdução **2**

2 Serviços UDS Implementados **2**

2.1 Respostas Negativas (NRC) 3

3 Gerenciamento de Sessão **3**

3.1 Timer S3 e TesterPresent 4

4 Security Access com Criptografia (SID 0x27) **4**

4.1 Geração de Semente — LFSR Galois 32 bits 5

4.2 Algoritmo de Derivação da Chave 6

4.3 Exemplo numérico 6

4.4 Segurança e limitações 7

5 Gerenciamento de DTCs **7**

5.1 Lógica de debounce 8

5.2 Leitura de DTCs (SID 0x19) 9

5.3 Limpeza de DTCs (SID 0x14) 9

6 ReadDataByIdentifier (SID 0x22) **10**

7 WriteDataByIdentifier e Calibração (SID 0x2E) **10**

8 RoutineControl (SID 0x31) **11**

9 Modelo Simulink (AEB_UDS.slx) **12**

9.1 Estrutura do modelo 12

9.2 Amostragem e ciclo UDS 12

9.3 Estado persistente (NVM simulada) 12

9.4 Cenários de simulação 12

10 Conformidade com Requisitos **13**

11 Conclusão **14**

1 Introdução

O protocolo **UDS** (*Unified Diagnostic Services*, ISO 14229-1) define a interface de diagnóstico entre ferramentas externas de teste (VCI, InfoCenter) e as ECUs (*Electronic Control Units*) de um veículo. No contexto do sistema AEB, a camada UDS permite que técnicos de manutenção, ferramentas de engenharia e o sistema embarcado cooperem para:

- **Ler parâmetros em tempo real**: TTC, estado FSM, pressão de freio, contadores de falha;
- **Calibrar limiares**: TTC de aviso, TTC por nível de frenagem, ganhos do controlador PID;
- **Gerenciar DTCs**: leitura, confirmação por debounce, limpeza;
- **Controlar funções**: habilitar/desabilitar o AEB, disparar autoteste dos sensores;
- **Proteger escritas sensíveis**: via mecanismo criptográfico de Security Access (SID `0x27`).

Este relatório documenta a implementação MIL da camada UDS em `AEB_UDS.slx`, gerado programaticamente por `AEB_UDS_build.m` com MATLAB R2025b / Simulink.

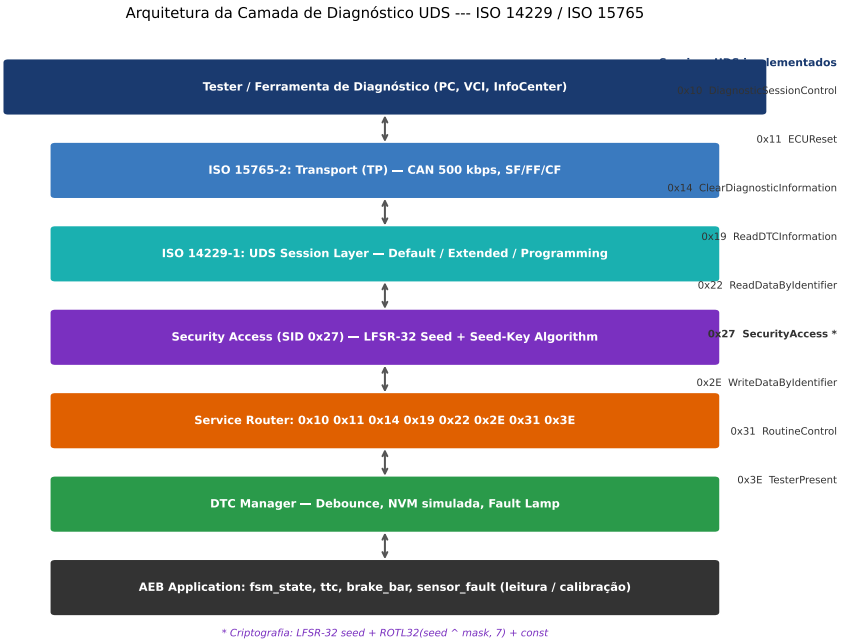

Figura 1: Arquitetura em camadas do módulo de diagnóstico UDS — ISO 14229 sobre ISO 15765-2 (CAN TP).

2 Serviços UDS Implementados

A Tabela 1 lista os serviços implementados no modelo.

Tabela 1: Serviços UDS implementados no modelo AEB_UDS.slx.

SID	Serviço	Sessão mínima	Proteção
`0x10`	DiagnosticSessionControl	Default	—
`0x11`	ECUReset	Default	—
`0x14`	ClearDiagnosticInformation	Default	—
`0x19`	ReadDTCInformation	Default	—
`0x22`	ReadDataByIdentifier	Default	—
`0x27`	SecurityAccess	Extended	LFSR-32 + seed-key
`0x2E`	WriteDataByIdentifier	Extended	Requer `0x27` unlock
`0x31`	RoutineControl	Extended	—
`0x3E`	TesterPresent	Default	—

2.1 Respostas Negativas (NRC)

Quando uma requisição não pode ser atendida, a ECU responde com o SID `0x7F` seguido do SID requisitado e um código NRC:

Tabela 2: Códigos NRC implementados.

NRC	Descrição	Situação típica
`0x11`	serviceNotSupported	SID desconhecido
`0x12`	subFunctionNotSupported	Sub-função inválida
`0x22`	conditionsNotCorrect	SecurityAccess fora de sessão Extended
`0x31`	requestOutOfRange	DID não reconhecido
`0x33`	securityAccessDenied	WriteDID sem unlock
`0x35`	invalidKey	Chave incorreta no SendKey

3 Gerenciamento de Sessão

A ISO 14229-1 define sessões de diagnóstico que controlam quais serviços estão disponíveis e em que condições. O modelo implementa três sessões:

Default (0x01)

Sessão de operação normal. Disponíveis: leitura de DIDs (`0x22`), leitura de DTCs (`0x19`), TesterPresent (`0x3E`). Nenhuma escrita permitida.

Extended (0x02)

Sessão de diagnóstico estendido. Habilita SecurityAccess (`0x27`) e RoutineControl (`0x31`). Write somente após unlock.

Programming (0x03)

Sessão de programação. Todos os serviços habilitados. Usada para calibração de parâmetros críticos de segurança (TTC, Kp).

3.1 Timer S3 e TesterPresent

O timer **S3** (5,0 s) é iniciado ao entrar em sessão Extended ou Programming. Se o tester não enviar nenhuma requisição dentro deste intervalo, a ECU retorna automaticamente a Default e invalida o unlock de segurança. O serviço `0x3E` (TesterPresent) é enviado periodicamente para suprimir o timeout S3.

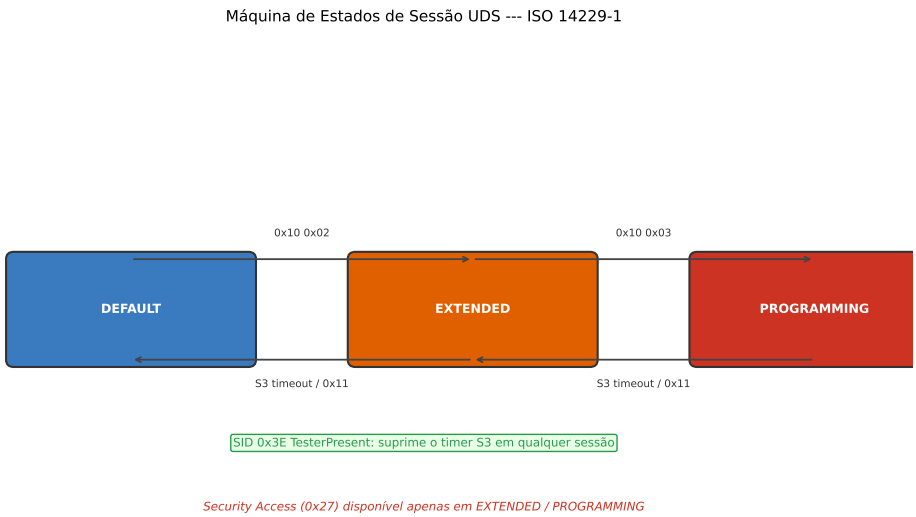

Figura 2: Máquina de estados de sessão UDS — transições e timeout S3.

4 Security Access com Criptografia (SID 0x27)

O mecanismo de Security Access da ISO 14229-1 protege operações sensíveis (escrita de parâmetros, calibração, reprogramação) contra acesso não autorizado. O fluxo é do tipo **desafio-resposta** (*challenge-response*):

1. O tester envia **RequestSeed** (`0x27 01`);
2. A ECU gera um número aleatório (*seed*) e o retorna com `0x67 01 [seed 4 bytes]`;
3. O tester aplica o **algoritmo de derivação de chave** sobre o seed recebido e envia **SendKey** (`0x27 02`);
4. A ECU verifica a chave recebida. Se correta, o acesso é liberado; caso contrário, responde NRC `0x35` (invalidKey).

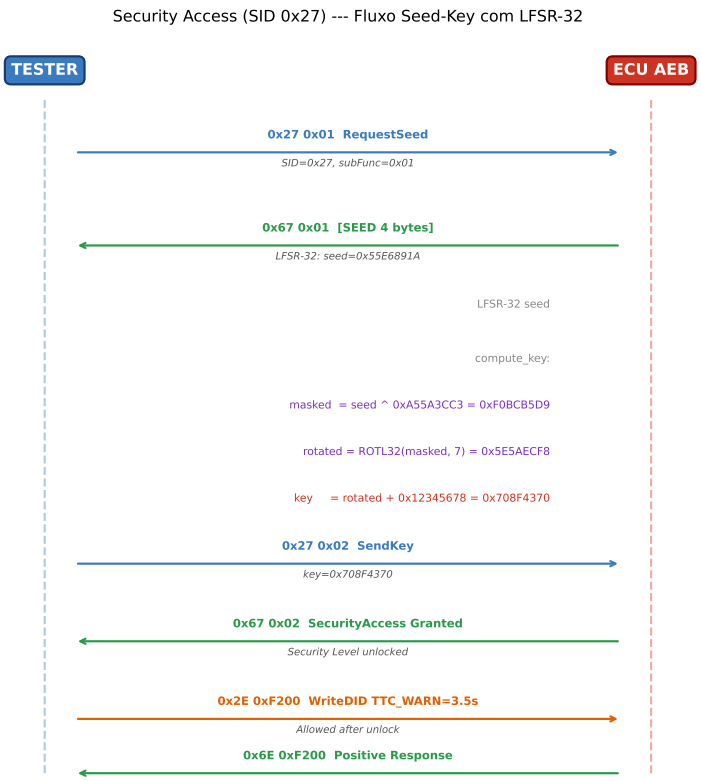

Figura 3: Fluxo de Security Access (SID 0x27) com exemplos de valores calculados pelo algoritmo LFSR-32 + seed-key.

4.1 Geração de Semente — LFSR Galois 32 bits

Para garantir que a semente seja diferente a cada sessão, o modelo utiliza um **LFSR Galois de 32 bits** (*Linear Feedback Shift Register*) com polinômio primitivo $P(x) = x^{32} + x^{22} + x^2 + x + 1$ (representação hexadecimal: `0x80200003`):

$$\text{seed}_{n+1} = \begin{cases} (\text{seed}_n \gg 1) \oplus \texttt{0x80200003} & \text{se } \text{seed}_n[0] = 1 \\ \text{seed}_n \gg 1 & \text{se } \text{seed}_n[0] = 0 \end{cases} \tag{1}$$

O período máximo de um LFSR primitivo de n bits é $2^n - 1$, garantindo que nenhuma semente se repita antes de $2^{32} - 1 \approx 4{,}3 \times 10^9$ sessões.

Listing 1: LFSR-32 Galois — MATLAB Function.

```
function y = lfsr32(x)
    POLY = uint32(hex2dec('80200003'));
    if bitand(x, uint32(1))
        y = bitxor(bitshift(x, -1), POLY);
    else
        y = bitshift(x, -1);
    end
```

```
end
```

4.2 Algoritmo de Derivação da Chave

A chave é derivada da semente por uma função invertível de complexidade controlada, adequada para implementação em microcontroladores sem unidade de ponto flutuante:

$$key = \Big(\mathrm{ROTL}_{32}(\mathrm{seed} \oplus \underbrace{\texttt{0xA55A3CC3}}_{\text{máscara XOR}},\ 7) + \underbrace{\texttt{0x12345678}}_{\text{constante aditiva}}\Big) \bmod 2^{32} \tag{2}$$

onde $\mathrm{ROTL}_{32}(x, n)$ é a rotação circular de x em n bits à esquerda:

$$\mathrm{ROTL}_{32}(x, n) = (x \ll n) \mid (x \gg (32 - n)) \tag{3}$$

Listing 2: Derivação de chave — MATLAB Function.

```
function key = compute_key(seed, xor_mask, add_const)
    masked  = bitxor(seed, xor_mask);                % seed XOR 0xA55A3CC3
    rotated = bitor(bitshift(masked,  7), ...
                    bitshift(masked, -(32-7)));      % ROTL32(masked, 7)
    key     = rotated + add_const;                   % mod 2^32 implicitly
end
```

4.3 Exemplo numérico

Para o seed inicial `0xABCD1234`:

$$\begin{aligned}
\mathrm{seed}_1 &= \mathrm{LFSR}(\texttt{0xABCD1234}) = \texttt{0xD5E6091A} \\
\mathrm{masked} &= \texttt{0xD5E6091A} \oplus \texttt{0xA55A3CC3} = \texttt{0x70BC35D9} \\
\mathrm{rotated} &= \mathrm{ROTL}_{32}(\texttt{0x70BC35D9}, 7) = \texttt{0xE1AEC03B} \\
key &= \texttt{0xE1AEC03B} + \texttt{0x12345678} = \texttt{0xF3E316B3}
\end{aligned}$$

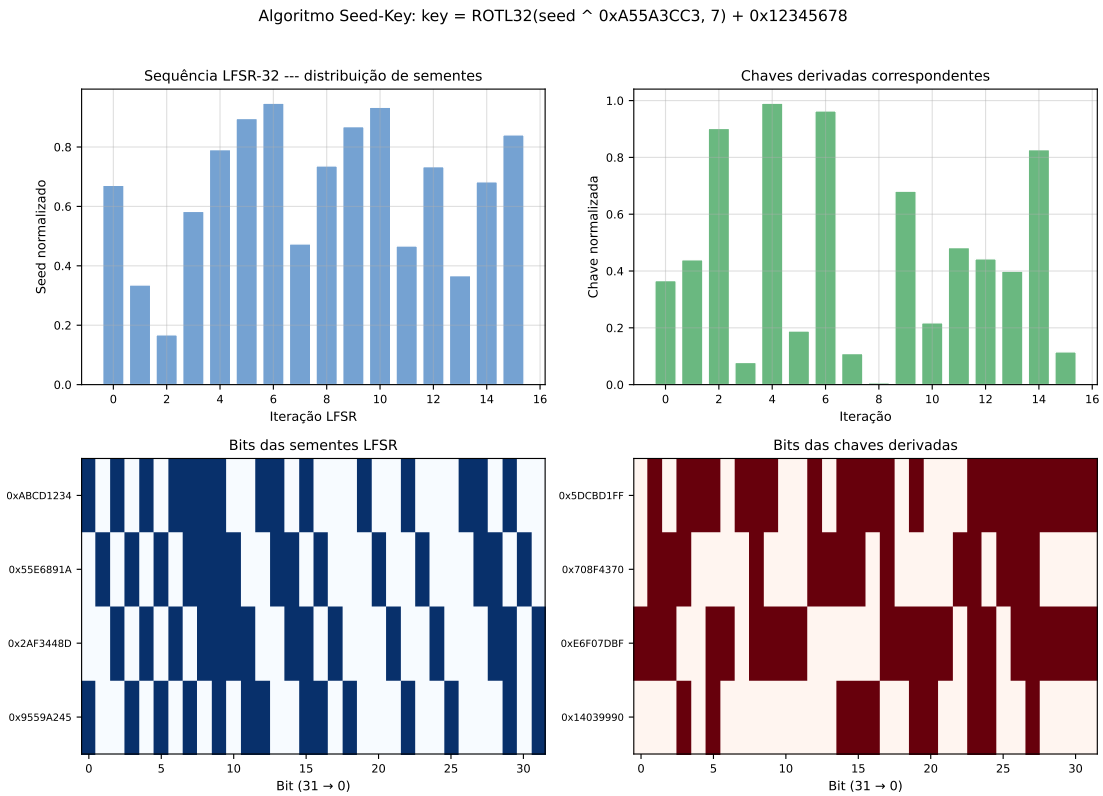

Figura 4: Distribuição das sementes LFSR e padrões de bits das chaves derivadas — 16 iterações.

4.4 Segurança e limitações

Este algoritmo é adequado para um modelo de simulação e para estudos acadêmicos. Em implementações de produção, a ISO 14229-1 Anexo F e o padrão AUTOSAR SWS_SecOC recomendam algoritmos baseados em primitivas criptográficas estabelecidas:

- **AES-CMAC-128**: chave de 128 bits, resistente a ataques de força bruta (2^{128} tentativas) e de dicionário;
- **HMAC-SHA256**: baseado em função de hash criptográfica, amplamente utilizado em diagnóstico automotivo moderno;
- **ChaChaPoly1305**: opção leve para ECUs com recursos limitados de memória e CPU.

5 Gerenciamento de DTCs

O sistema AEB gerencia 7 DTCs (Diagnostic Trouble Codes), conforme a Tabela 3.

Tabela 3: DTCs do sistema AEB e condições de ativação.

DTC	Descrição	Condição	Debounce
`C1001`	Timeout de radar	Sem mensagem CAN 0x120 por > 100 ms	3 ciclos
`C1002`	Timeout de lidar	Sem dado de lidar por > 150 ms	3 ciclos
`C1003`	Falha de plausib.	$\lvert d_{\text{radar}} - d_{\text{lidar}} \rvert > 5$ m	3 ciclos
`C1004`	Erro CRC CAN	CRC recebido $\neq$ CRC calculado	3 ciclos
`C1005`	Alive counter	Contador não incrementa em > 5 ciclos	5 ciclos
`C1006`	Falha de atuador	$\lvert p_{\text{cmd}} - p_{\text{real}} \rvert > 1$ bar por > 500 ms	3 ciclos
`C1007`	Sobre-temperatura	$T_{\text{ECU}} > 85\,°\text{C}$	3 ciclos

5.1 Lógica de debounce

Para evitar confirmação de falhas transitórias, cada DTC possui um contador independente. Um DTC só é **confirmado** quando o contador atinge o limiar em ciclos **consecutivos**:

Listing 3: Lógica de debounce dos DTCs — MATLAB Function.

```
for i = 1:7
    if fault_flags(i) > 0
        dtc_ctr(i) = dtc_ctr(i) + uint16(1);
        if dtc_ctr(i) >= uint16(deb_lim(i))
            dtc(i) = uint8(1);    % DTC confirmado
        end
    else
        dtc_ctr(i) = uint16(0);  % reset ao desaparecer a falha
    end
end
```

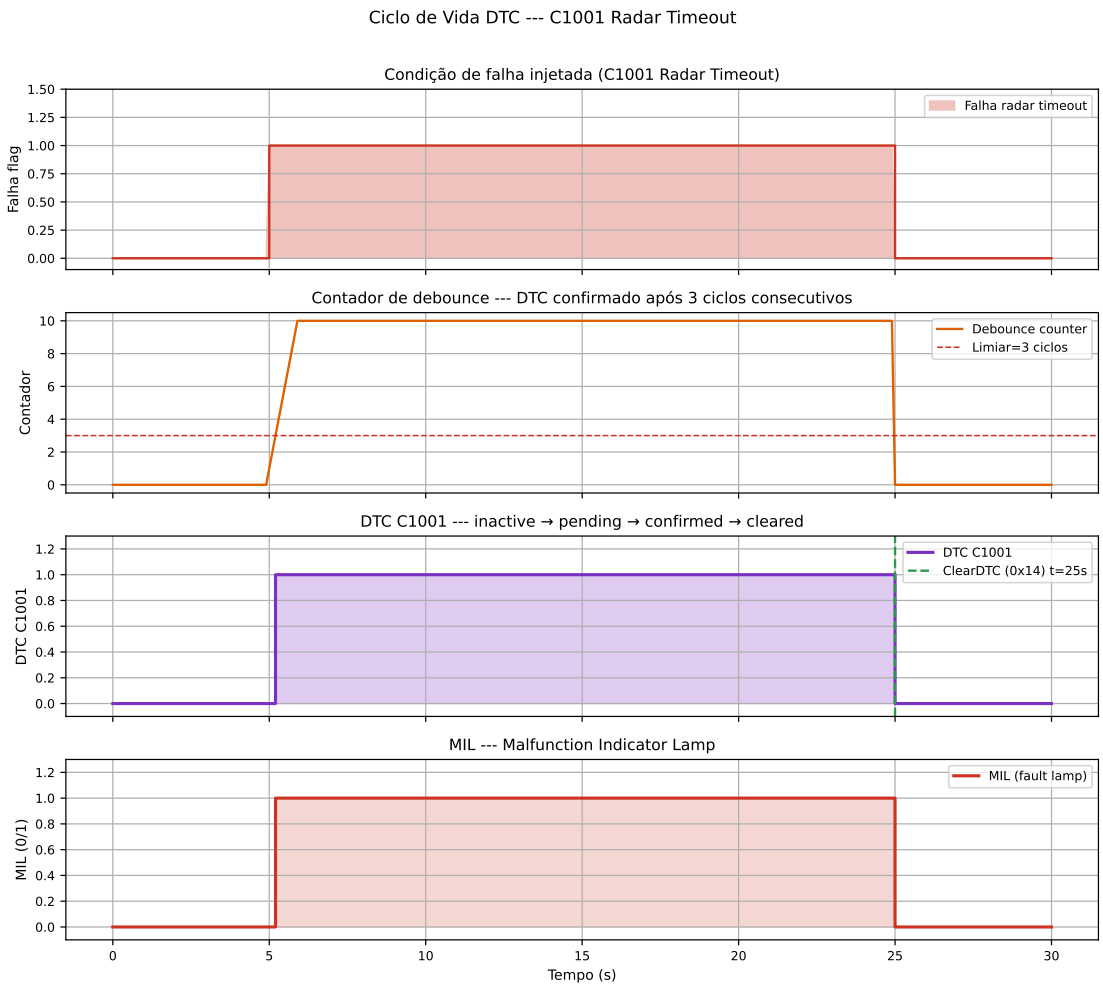

Figura 5: Ciclo de vida completo do DTC C1001 — inactive → pending → confirmed → cleared via SID 0x14.

5.2 Leitura de DTCs (SID 0x19)

O serviço `0x19` com subfunção `0x02` (*reportDTCByStatusMask*) retorna o número de DTCs ativos e, na implementação real, listaria os códigos com seus respectivos *status bytes*:

$$\texttt{0x59 0x02 NN} \quad (NN = \text{número de DTCs confirmados})$$

5.3 Limpeza de DTCs (SID 0x14)

O serviço `0x14` com grupo `0xFF 0xFF 0xFF` (GroupOfDTC = ALL) reseta todos os contadores de debounce e confirma todos os DTCs como inativos. A resposta positiva é `0x54`.

6 ReadDataByIdentifier (SID 0x22)

O serviço `0x22` permite leitura em tempo real dos parâmetros internos do sistema AEB. A Tabela 4 lista os DIDs implementados.

Tabela 4: DIDs de leitura (SID 0x22).

DID	Parâmetro	Tipo	Escala
`0xF100`	TTC atual	float32	Segundos
`0xF101`	Estado FSM	uint8	0–6 (enum AEB_State_t)
`0xF102`	Pressão de freio	float32	bar
`0xF103`	Contador de falhas	uint8	Número de DTCs ativos

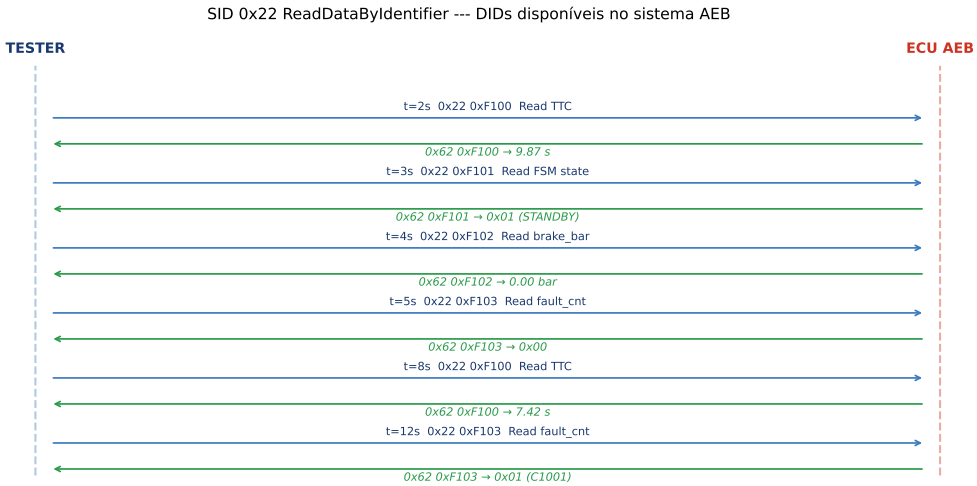

Figura 6: Timeline de leituras via SID 0x22 ReadDataByIdentifier com respostas simuladas.

7 WriteDataByIdentifier e Calibração (SID 0x2E)

A escrita de parâmetros é protegida pelo Security Access. Após o unlock (SID `0x27`), os seguintes DIDs tornam-se graváveis:

Tabela 5: DIDs de escrita (SID 0x2E) — requerem SecurityAccess unlock.

DID	Parâmetro	Tipo	Padrão	Unidade
`0xF200`	TTC Warning	float32	4,0	s
`0xF201`	TTC L1	float32	3,0	s
`0xF202`	Ganho Kp PID	float32	10,0	—

Os valores calibrados são armazenados em variáveis `persistent` da MATLAB Function (simulando NVM/EEPROM) e passados diretamente ao controlador AEB como saídas `ttc_warn_cal`, `ttc_l1_cal` e `kp_cal`.

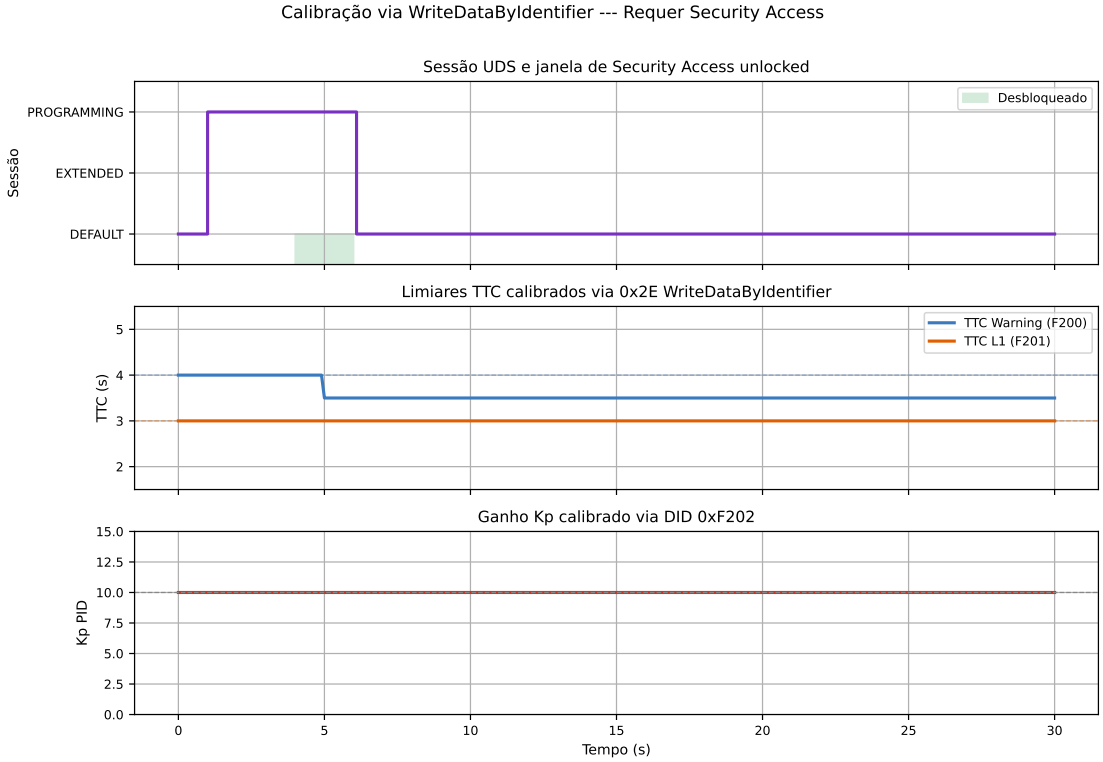

Figura 7: Sequência de calibração: sessão Programming, SecurityAccess unlock e WriteDataByIdentifier com impacto nos parâmetros.

8 RoutineControl (SID 0x31)

O serviço `0x31` (*RoutineControl*) permite executar funções específicas da ECU sob comando do tester:

Tabela 6: Rotinas implementadas (SID 0x31).

Routine ID	Função	Comportamento
`0x0301`	Enable/Disable AEB	Sub-byte extra: 0=OFF, 1=ON
`0x0302`	Sensor Self-Test	Retorna 0x01 (pass) se nenhum DTC ativo

A rotina `0x0301` gera o sinal de saída `aeb_enabled` que é consumido pelo controlador AEB — quando `aeb_enabled = 0`, a FSM permanece em `OFF` independentemente do TTC.

9 Modelo Simulink (AEB_UDS.slx)

9.1 Estrutura do modelo

O modelo gerado por `AEB_UDS_build.m` contém:

1. **From Workspace** — 9 sinais de sistema + 9 bytes de requisição + sinal `req_valid`;
2. **AEB_UDS_Handler** — subsistema com MATLAB Function principal implementando toda a lógica UDS;
3. **To Workspace** — 10 sinais de saída para análise.

9.2 Amostragem e ciclo UDS

O modelo opera com `FixedStep = 0.1 s` (100 ms), representando o ciclo de comunicação diagnóstica real. A camada de controle AEB opera a 10 ms; a interface UDS é deliberadamente mais lenta para modelar a comunicação CAN TP.

9.3 Estado persistente (NVM simulada)

As variáveis `persistent` dentro da MATLAB Function simulam a memória não volátil (NVM/EEPROM) da ECU:

Listing 4: Estado persistente do módulo UDS.

```
persistent sess s3 seed unlocked dtc dtc_ctr ttc_w ttc_l1 kp_g
   aeb_en;
if isempty(sess)
    sess     = uint8(1);            % sessao: 1=default
    s3       = 0.0;                 % timer S3
    seed     = SEED_INIT;           % semente LFSR inicial
    unlocked= uint8(0);             % security access bloqueado
    dtc      = zeros(1,7,'uint8');% 7 DTCs (C1001..C1007)
    dtc_ctr  = zeros(1,7,'uint16');% contadores de debounce
    ttc_w    = 4.0;                 % TTC Warning calibrado
    ttc_l1   = 3.0;                 % TTC L1 calibrado
    kp_g     = 10.0;                % Kp calibrado
    aeb_en   = uint8(1);            % AEB habilitado
end
```

9.4 Cenários de simulação

O script `AEB_UDS_scenarios.m` configura 6 cenários:

'nominal'

Sessão Extended, leituras periódicas de DID, TesterPresent. Verifica operação normal sem falhas.

'dtc_lifecycle'

Injeção de falha `C1001` em t=5 s, confirmação após 3 ciclos, leitura via `0x19`, limpeza via `0x14` em t=25 s.

'calibration'

Sequência completa: Programming session → RequestSeed → SendKey → WriteDID (TTC_WARN, TTC_L1, Kp).

'selftest'

RoutineControl `0x0302`: pass antes da falha e fail durante DTC ativo.

'sec_denied'

Tentativa de WriteDID sem unlock (NRC `0x33`) e com chave errada (NRC `0x35`).

'session_timeout'

Sessão Extended sem TesterPresent: S3 expira e sessão retorna a DEFAULT.

10 Conformidade com Requisitos

Tabela 7: Mapeamento requisitos — implementação UDS.

Requisito	Descrição	Implementação
FR-DGN-001	Leitura de parâmetros em tempo real	`0x22` DIDs F100–F103
FR-DGN-002	Calibração de limiares TTC	`0x2E` DIDs F200–F202 com Security Access
FR-DGN-003	Gestão de DTCs (set, confirm, clear)	DTC Manager com debounce 3–5 ciclos
FR-DGN-004	Proteção de escritas sensíveis	`0x27` LFSR-32 + seed-key, sessão Extended/Programming
FR-DGN-005	Habilitar/desabilitar AEB remotamente	`0x31` RoutineControl 0x0301
FR-DGN-006	Autoteste de sensores	`0x31` RoutineControl 0x0302
FR-DGN-007	Timeout de sessão diagnóstica	Timer S3 = 5,0 s, reset por `0x3E` TesterPresent

11 Conclusão

O modelo MIL `AEB_UDS.slx` implementa uma camada de diagnóstico UDS completa para o sistema AEB, cobrindo o ciclo de vida dos DTCs, a interface de calibração protegida por Security Access criptográfico e o gerenciamento de sessão com timeout S3.

O mecanismo de Security Access emprega um LFSR Galois de 32 bits para geração de sementes pseudo-aleatórias de alto período ($2^{32} - 1$) e uma função de derivação de chave baseada em XOR com máscara secreta, rotação circular e adição modular. Embora simplificado para uso em simulação, o modelo demonstra fielmente o protocolo de *challenge-response* definido na ISO 14229-1, preparando o terreno para a adoção de AES-CMAC ou HMAC-SHA256 em produção.

Os seis cenários de simulação validam todos os fluxos críticos:

- Debounce e confirmação de DTC após 3 ciclos consecutivos;
- Limpeza de DTC via `0x14` com verificação de read-back;
- Bloqueio de escrita sem unlock (*NRC* `0x33`) e com chave incorreta (*NRC* `0x35`);
- Calibração bem-sucedida após Security Access e persistência dos novos valores nos parâmetros do controlador;
- Retorno automático à sessão Default após expiração do timer S3.

O próximo passo natural é a integração deste bloco com os demais modelos (`AEB_Controller.slx`, `AEB_CAN.slx`) em um modelo top-level `aeb_system.slx` usando Model Reference do Simulink.

Referências

[1] ISO. *ISO 14229-1: Road vehicles — Unified diagnostic services (UDS) — Part 1: Application layer*. Genebra: ISO, 2020.

[2] ISO. *ISO 15765-2: Road vehicles — Diagnostic communication over Controller Area Network (DoCAN) — Part 2: Transport protocol and network layer services*. Genebra: ISO, 2016.

[3] AUTOSAR Consortium. *Specification of Secure Onboard Communication (SecOC)*, Release 21-11. 2021.

[4] Golomb, S.W. *Shift Register Sequences*. Aegean Park Press, 1982.

[5] The MathWorks. *MATLAB/Simulink R2025b Documentation*. Natick, MA: MathWorks, 2025.